Propuesta de Arquitectura de Integración Modernización Bancaria

Documento Técnico
Arquitectura Empresarial

Autor: Gabriel Borja Moscoso

Tabla de Contenido

1. Resumen Ejecutivo
2. Arquitectura de Alto Nivel
3. Estrategia Multicore
4. Patrones de Integración
5. Seguridad y Cumplimiento
6. Alta Disponibilidad y DR
7. Gobierno de APIs
8. Plan de Migración

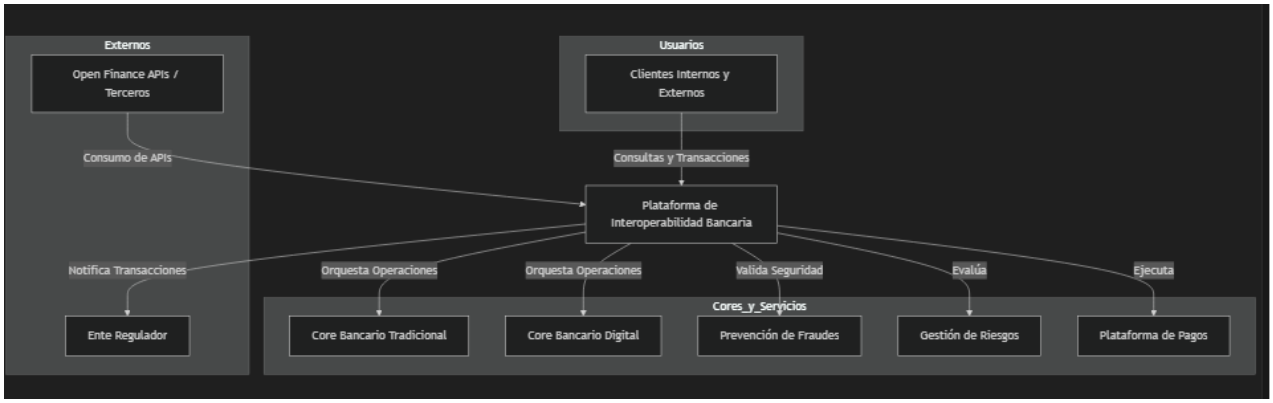
1. Resumen Ejecutivo

La presente solución propone una arquitectura de integración moderna, escalable y segura para la coexistencia de un sistema bancario tradicional (Legacy) con un nuevo ecosistema digital cloud-native. Basada en el estándar industrial BIAN, se implementa una capa de servicios de dominio que abstrae la complejidad de los núcleos bancarios mediante una estrategia Multicore inteligente. La arquitectura prioriza la seguridad mediante el cumplimiento de la LOPDP con protocolos OAuth2/OIDC y mTLS, garantiza la alta disponibilidad regional y facilita una migración de bajo riesgo operativo a través del patrón Strangler Fig.

2. Arquitectura de Alto Nivel (C4)

2.1 Contexto del Sistema (Nivel 1)

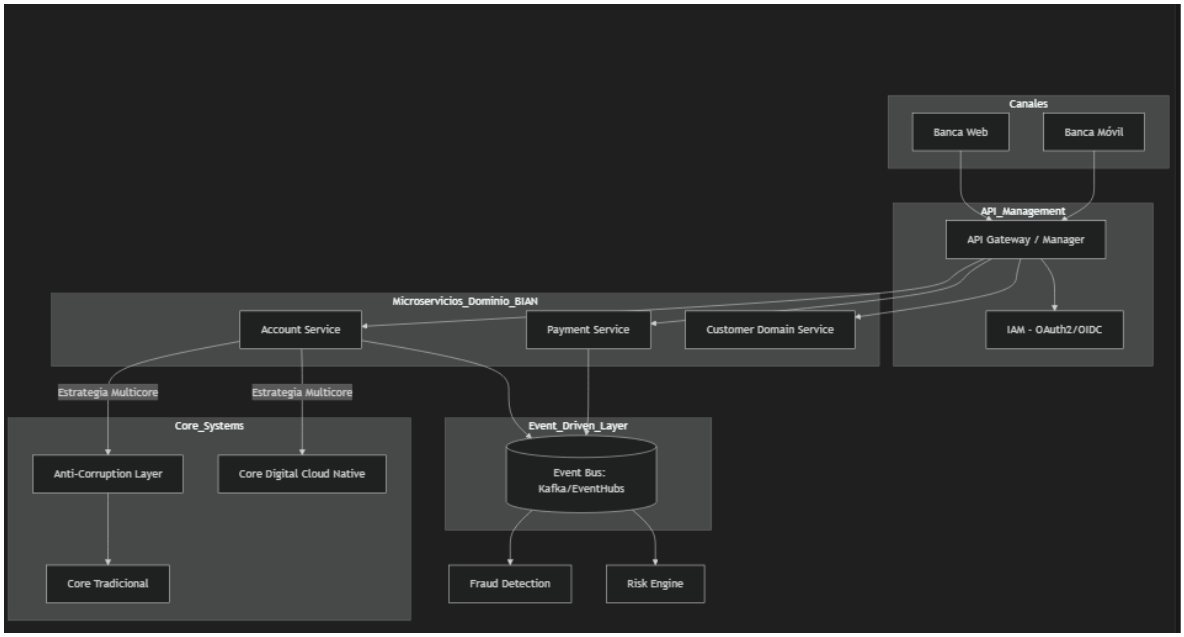
La plataforma actúa como el orquestador central que habilita servicios de banca web, móvil y APIs de Open Finance, integrando de forma transparente los Cores bancarios y sistemas críticos como Prevención de Fraudes y Gestión de Riesgos.



2.2 Contenedores (Nivel 2)

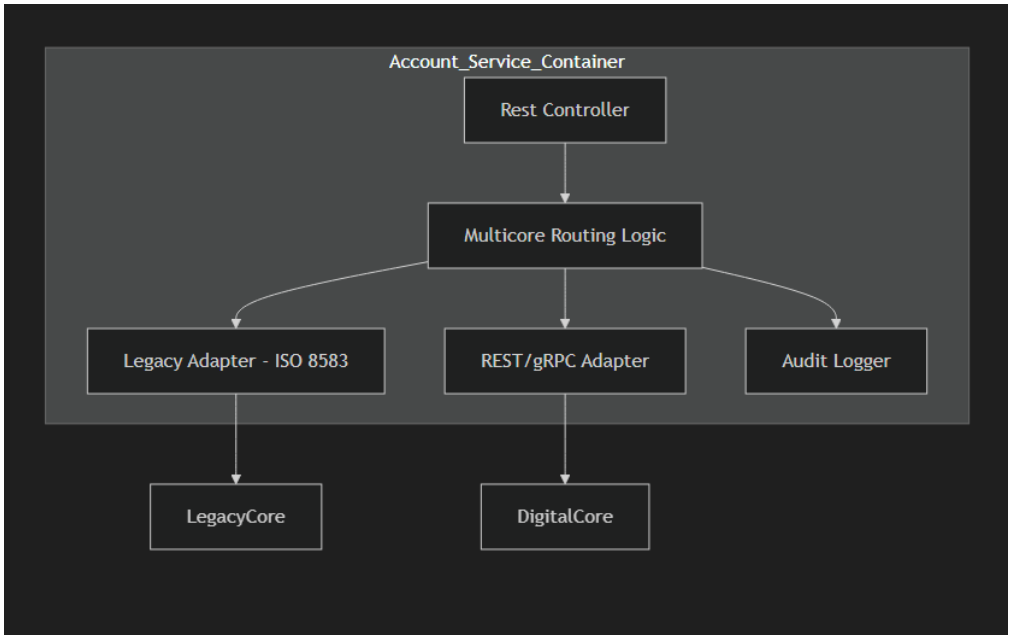
Se detalla la infraestructura de microservicios segmentada:

- Capa de API Management: Control de acceso, cuotas y seguridad perimetral.
- Servicios de Dominio BIAN: Servicios desacoplados (Cuentas, Pagos, Clientes) que consumen de ambos Cores.
- Capa de Eventos: Uso de un Bus de Eventos (Kafka) para procesamiento asíncrono y detección de fraudes en tiempo real.
- Capa de Adaptación (ACL): Protege la lógica moderna de la rigidez del Core Tradicional.



2.3 Componentes (Nivel 3)

Detalle del motor de ruteo dentro de los microservicios, permitiendo decidir en tiempo real (basado en segmentación de clientes o producto) si una transacción se procesa en el Core Tradicional o en el Digital.



3. Estrategia Multicore

La coexistencia de núcleos se gestiona mediante una capa de abstracción técnica. BIAN (Banking Industry Architecture Network) se utiliza para definir los dominios de servicio, asegurando que el banco hable un "lenguaje estándar" independientemente de si el backend es un Mainframe o una nube distribuida..

4. Patrones de Integración

- Anti-Corruption Layer (ACL): Para traducir el modelo de datos de sistemas antiguos a APIs modernas.
- Event-Driven Architecture (EDA): Mejora la latencia y permite que sistemas de fraude reaccionen sin bloquear la transacción principal.
- Backends for Frontends (BFF): Optimiza la entrega de datos para aplicaciones móviles y web.
- Stack: Kubernetes, Apache Kafka, API Gateway (Azure/AWS), bases de datos PostgreSQL y Redis.

5. Seguridad y Cumplimiento

- Identidad: Implementación de IAM (Identity & Access Management) centralizado con OAuth2 y OpenID Connect.
- Protección de Datos: Cifrado AES-256 en reposo, mTLS para comunicación interna y tokenización de datos sensibles para cumplimiento estricto de la LOPDP.
- Cumplimiento: Logs inmutables y auditoría en tiempo real mediante el stack ELK..

6. Alta Disponibilidad y Recuperación

- Disponibilidad: Despliegue en múltiples zonas de disponibilidad (Multi-AZ) con balanceo de carga global.
- DR: Estrategia de recuperación ante desastres en región secundaria (Warm Standby).
- Métricas: RTO (Tiempo de Recuperación) < 15 minutos y RPO (Punto de Recuperación) < 5 minutos.

7. Gobierno de APIs

- Ciclo de Vida: Uso de un API Portal para documentación y control de versiones (SemVer).
- Service Mesh: Monitoreo y trazabilidad distribuida de todas las llamadas entre servicios para debugging rápido.

8. Plan de Migración

La migración no será un "Big Bang". Se utilizará el patrón del "Higuera Estranguladora":

1. Interceptar rutas en el Gateway.
2. Crear nuevos productos en el Core Digital.
3. Mover datos de clientes antiguos de forma gradual.
4. Apagar módulos del Core Tradicional una vez validados en el nuevo entorno.

.